Node.js MySQL Demo Project

Overview

This project demonstrates a simple full-stack Node.js application using:

- Node.js
- Express.js
- MySQL
- EJS Template Engine

The application connects directly to a MySQL database, fetches user data, and dynamically renders it on the UI.

The goal of this project is to understand:

- Backend routing
- Database integration
- Dynamic server-side rendering
- MVC-style architecture
- End-to-end request flow

Technology Stack

Technology	Purpose
Node.js	Backend Runtime
Express.js	Web Framework
MySQL	Relational Database
EJS	Server-Side Rendering
HTML/CSS	UI Layer
npm	Dependency Management

High-Level Architecture

```
Browser
   ↓
Node.js Server
   ↓
Express Routes
```

```
   ↓
MySQL Database
   ↓
EJS Rendering
   ↓
HTML Response
```

Project Architecture

```
Client Browser
      ↓
Express Route
      ↓
Controller Logic
      ↓
MySQL Query
      ↓
Database Response
      ↓
EJS Template
      ↓
Rendered HTML Page
```

Project Structure

```
node-mysql-demo
 ├ routes
 │ └ userRoutes.js
 ├ db
 │ └ db.js
 ├ views
 │ ├ index.ejs
 │ └ userDetail.ejs
 ├ public
 │ └ style.css
 ├ package.json
 ├ package-lock.json
 └ server.js
```

Application Features

The project includes:

- User Dashboard
- User Detail Page
- MySQL Integration
- Dynamic Route Parameters
- Server-Side Rendering
- Dynamic Database Queries

Database Design

Database Name

```
node_demo
```

Users Table

```sql
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    email VARCHAR(100),
    city VARCHAR(100)
);
```

Sample Data

```sql
INSERT INTO users (name, email, city) VALUES
('Rohit', 'rohit@test.com', 'Bhopal'),
('Amit', 'amit@test.com', 'Indore'),
('Sarah', 'sarah@test.com', 'Delhi');
```

Why MySQL?

MySQL is used as the application's primary data source.

Benefits:

- Structured data storage
- Relational data handling
- SQL query support
- Easy integration with Node.js

Backend Layer

server.js

Responsibilities:

- Starts Node.js server
- Configures Express
- Registers routes
- Configures EJS
- Exposes static files

Example

```
app.use("/", userRoutes);
```

Database Connection Layer

File

```
db/db.js
```

Responsibilities

- Establishes MySQL connection
- Provides DB access to routes
- Handles connection setup

Example

```
mysql.createConnection({
    host: "localhost",
    user: "root",
    password: "",
```

```
    database: "node_demo"
});
```

Routing Layer

File

```
routes/userRoutes.js
```

Responsibilities:

- Handles browser requests
- Executes database queries
- Sends data to UI

Home Route

Endpoint

```
/
```

SQL Query

```
SELECT * FROM users
```

Responsibilities

- Fetch all users
- Render dashboard page

User Detail Route

Endpoint

```
/user/:id
```

SQL Query

```
SELECT * FROM users WHERE id = ?
```

Responsibilities

- Read route parameter
- Fetch single user
- Render detail page

Dynamic Route Parameters

Example

```
req.params.id
```

Used for:

```
/user/1
/user/2
```

Frontend Rendering Layer

EJS Templates

EJS is used for server-side rendering.

Responsibilities:

- Generate dynamic HTML
- Display database data
- Create reusable UI rendering

Dashboard Page

File

```
views/index.ejs
```

Displays:

- User List
- Email
- City
- User Links

User Detail Page

File

```
views/userDetail.ejs
```

Displays:

- User ID
- Name
- Email
- City

Styling Layer

File

```
public/style.css
```

Responsibilities:

- UI styling
- Table formatting
- Layout improvements

End-to-End Flow

```
Browser Request
      ↓
Express Route
      ↓
MySQL Query
      ↓
Query Result
      ↓
EJS Template Rendering
      ↓
HTML Response
      ↓
Browser Display
```

Example Application Flow

Step 1

Open:

```
http://localhost:3000
```

Step 2

Route executes:

```
SELECT * FROM users
```

Step 3

Users displayed on dashboard.

Step 4

User clicks profile link.

Step 5

Route:

```
/user/1
```

executes:

```
SELECT * FROM users WHERE id = 1
```

Step 6

User detail page displayed.

MVC Style Mapping

Node.js Layer	Spring Boot Equivalent
Express Route	Controller
MySQL Query	Repository
EJS	JSP / Thymeleaf
server.js	Main Application
db.js	Datasource Configuration

Advantages of This Architecture

- Clear separation of concerns
- Easy to understand
- Lightweight backend
- Dynamic UI rendering
- Direct database integration
- Simple MVC-style structure

Real-World Use Cases

This architecture can be extended for:

- User Management Systems
- Admin Dashboards
- CRM Applications
- Reporting Systems
- Internal Tools
- CRUD Applications

Future Enhancements

The project can be extended with:

- REST APIs
- React Frontend
- Angular Frontend
- Authentication
- JWT Security
- CRUD Operations
- Pagination
- Search Functionality
- Kafka Integration
- Redis Cache
- Docker Deployment

README

Node.js MySQL Demo Application

Introduction

This project demonstrates a Node.js application that connects directly to MySQL and dynamically displays data using EJS.

Features

- Display all users
- User detail page
- Dynamic routing

- MySQL integration
- Server-side rendering

Prerequisites

Install:

- Node.js
- MySQL
- VS Code / IntelliJ IDEA
- npm

Database Setup

Step 1

Open MySQL.

Step 2

Run:

```
CREATE DATABASE node_demo;

USE node_demo;

CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    email VARCHAR(100),
    city VARCHAR(100)
);

INSERT INTO users (name, email, city) VALUES
('Rohit', 'rohit@test.com', 'Bhopal'),
('Amit', 'amit@test.com', 'Indore'),
('Sarah', 'sarah@test.com', 'Delhi');
```

Install Dependencies

Run:

```
npm install
```

Configure Database

Open:

```
db/db.js
```

Update:

```
password: "YOUR_PASSWORD"
```

Run Application

Command

```
npm start
```

Application URL

```
http://localhost:3000
```

Available Routes

Route	Description
/	User Dashboard
/user/:id	User Detail Page

Example

Dashboard

```
http://localhost:3000
```

User Detail

```
http://localhost:3000/user/1
```

Common Issues

MySQL Access Denied

Check:

- Username
- Password
- MySQL running status

Port Already Used

If port 3000 is busy:

Change:

```
const PORT = 3001;
```

Cannot GET /

Check:

- Route registration
- Server running
- Correct URL

---

package-lock.json

This file is automatically generated by npm.

Purpose:

- Locks dependency versions
- Ensures consistent installations
- Prevents dependency mismatch

Learning Outcome

This project helps understand:

- Node.js backend development
- Express routing
- MySQL integration
- Dynamic rendering
- MVC-style architecture
- Full-stack request flow

Conclusion

This demo project demonstrates how Node.js can connect directly to MySQL, retrieve structured data, and dynamically render it on the frontend using EJS templates.